ENSEEIHT 2SN - Groupe M2

Projet Modélisation Géométrique - Banc de poissons

Rapport de :
Elena FLEURY
Yan RABEFANIRAKA

Rendu le 18 Mai 2026

Introduction

Le but de ce projet est de modéliser un banc de poisson dynamique en tant que système de particules, capable de se soumettre à des forces externes.

Nous étaient proposés cinq options de comportement du banc de poissons, avec le **comportement aléatoire** et l'**effet Trafalgar** obligatoires. Nous avons choisi d'implémenter dans un premier temps les cinq modèles, puis à partir de ceux-là nous avons établi notre propre système de banc de poissons. De plus, nous avons ajouté de quoi lui soumettre à un courant d'eau.

Dans ce rapport, nous étaierons donc l'implémentation des cinq premiers modèles, puis comment cela nous a permis de converger vers notre propre système que nous détaillerons. Ensuite, nous conclurons sur la qualité de la réalisation et discuterons des limitations actuelles ainsi que les améliorations qui pourraient donc les corriger.

Table des matières

1 Modélisation du banc de poissons **1**
1.1 Résultats des modèles 1 à 5 1
1.2 Notre modèle 2
1.2.1 Conglomérat de modèles 2
1.2.2 Résultats finaux 2

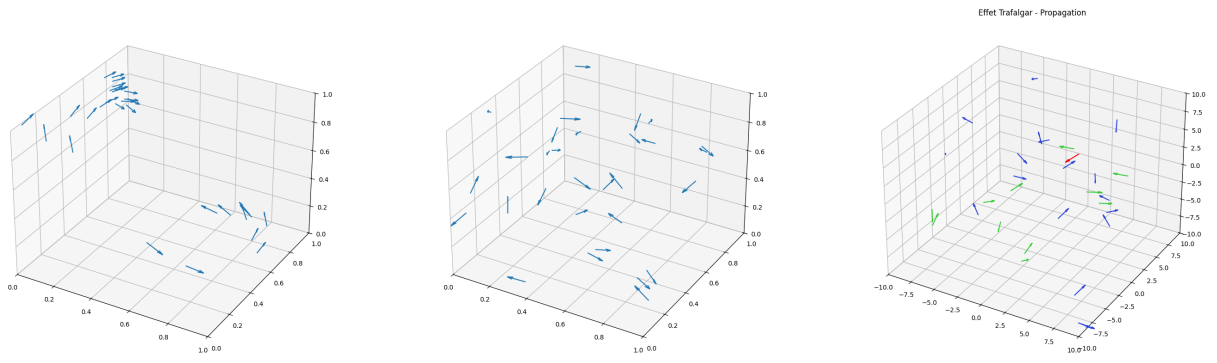
2 Conclusion **3**

Chapitre 1

Modélisation du banc de poissons

1.1 Résultats des modèles 1 à 5

— **Modèle 1 - Mouvement aléatoire (1.1a et 1.1b) :** Nous avons testé deux approches : La première [1] utilise un champ vectoriel 3D à oscillations périodiques projeté sur la surface d'une sphère. Bien qu'esthétique grâce à de jolies trajectoires courbées, elle créait malheureusement des puits (sinusoïdes sur $[0, 1]$) dans le champ 3D, où les poissons convergeaient et restaient bloqués régulièrement. La seconde consistait à initialiser des positions et vitesses aléatoires dans la grille 3D (avec inversion aux frontières). Cette dernière s'est avérée plus fonctionnelle, bien que moins agréable visuellement ; C'est elle qui servira de postulat de base aux modèles suivants.



(a) Mouvement aléatoire 1 (b) Mouvement aléatoire 2 (c) Effet Trafalgar

FIGURE 1.1 – Observation des modèles 1 et 2.

Pour les modèles suivants (précisons qu'ils sont indépendants les uns des autres), nous sommes partis de notre seconde implémentation du mouvement aléatoire pour l'initialisation et la gestion des collisions avec la bordure :

— **Modèle 2 - L'effet Trafalgar (1.1c) :** Dans ce modèle, nous initialisons un "leader" dont le comportement anormal (vitesse élevée) contamine périodiquement les voisins à proximité. Afin d'éviter une uniformisation du mouvement, nous appliquons aussi un léger bruit à cette altération de la vitesse, simulant ainsi un effet Trafalgar réaliste.

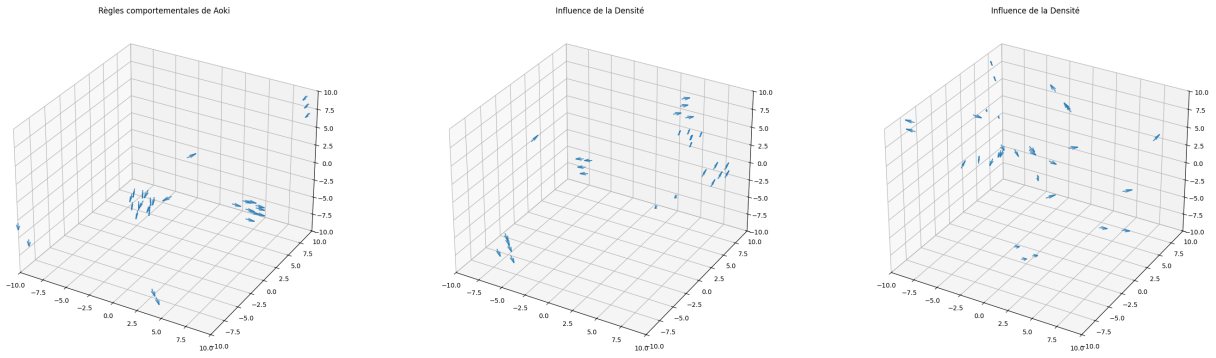
— **Modèle 3 - Règles comportementales d'Aoki (1.2a) :** Chaque poisson réagit selon trois échelles de voisinage exclusives : Répulsion (évitement proche), Alignement (moyenne distance), et Attraction (rapprochement lointain). L'utilisation d'un arbre *KDTree* (`scipy.spatial`) a permis d'éviter la complexité en $O(N^2)$ de deux boucles *for*. Ainsi, nous modélisons les règles principales de notre banc de poissons et observons des comportements de groupe cohérents.

— **Modèle 4 - Influence de la densité (1.2b) :** Partant du "Modèle 3", on réduit l'influence aux seuls **6 plus proches voisins** de chaque poisson afin de prendre en compte l'influence de la densité et donc d'obtenir une simulation plus réaliste.

— **Modèle 5 - Réseau d'influence et cône de vision (1.2c) :** Enfin, nous avons réduit cette influence à un cône de vision devant chaque poisson. En calculant simplement un produit scalaire, on détermine les

1. inspiré de cet exemple de la documentation Matlab

poissons visibles ainsi que l'échelle de voisinage à laquelle ils appartiennent. Cette implémentation nous permet de parfaire le réalisme de notre simulation.



(a) Règles comportementales d'Aoki (b) Influence de la densité (c) Réseau d'influence

FIGURE 1.2 – Observation des modèles 3 à 5.

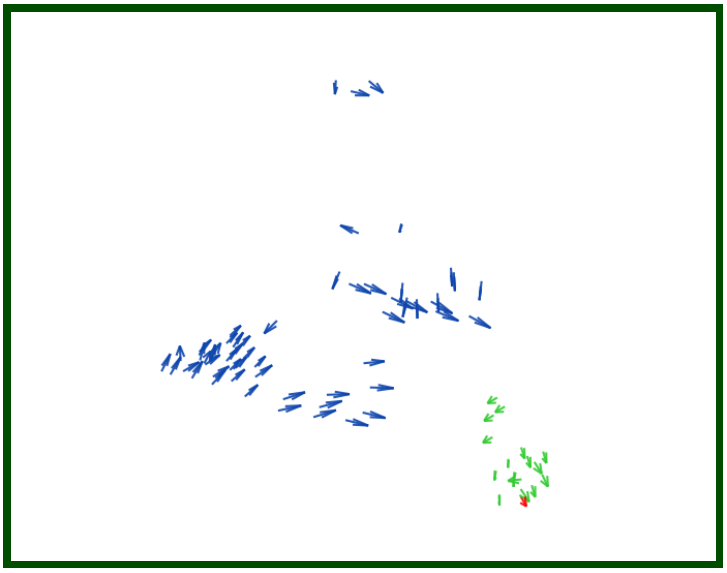
1.2 Notre modèle

1.2.1 Conglomérat de modèles

A partir des résultats de la section 1.1, nous avons établi notre système final comme **conglomérat des règles précédentes** : Le banc est régi par les règles comportementales d'Aoki limitées à 6 voisins stricts, et disrupté par l'injection périodique d'un "leader" rapide. Tout ceci est pondéré par un cône de vision limitant les différentes influences aux seuls voisins visibles frontalement. L'ajout d'une écoute de claviers [2] permet également à l'utilisateur de soumettre le système à des **courants marins**.

1.2.2 Résultats finaux

Ainsi, nous obtenons une simulation réaliste d'un banc de poissons grâce à toutes ces règles combinées. De plus on peut aussi soumettre le système à l'influence extérieure du courant marin. Voici le résultat final :



(a) Modèle final

FIGURE 1.3 – Observation du modèle final.

2. touches directionnelles avec *event listeners* de matplotlib

Chapitre 2

Conclusion

Ainsi, en implémentant à la suite les 5 modèles proposés par le sujet, allant du simple mouvement aléatoire aux règles comportementales d'Aoki puis au cône de vision du réseau d'influence, nous avons pu converger vers un modèle plus réaliste qui pioche les essentiels de ces prédécesseurs.

En effet notre modèle final nous satisfait bien, puisqu'il permet de réaliser/combiner plusieurs modèles de la littérature en système de particules avec un banc de poissons autonome, qui pour autant peut aussi subir des forces extérieures telles que le courant que nous avons implémenté. Ainsi, sur le plan du *comportement de groupe*, notre implémentation fait bien son travail.

Pour autant, plusieurs points pourraient être perfectionnés : tout d'abord l'implémentation en Python réduit grandement les performances à cause des boucles *for* utilisées. Ainsi, notre simulation est fluide pour notre petit nombre de poissons, mais un nombre plus conséquent (et réaliste) causerait une chute massive de fluidité. Aussi, nous partons du postulat que chaque poisson nage à la même vitesse constante. Cela cause non seulement une uniformisation artificielle de leur déplacement, mais aussi un manque d'inertie visible (accélération nulle) notamment quand ils changent de direction. Enfin, ce changement de direction étrange est aussi à noter pour les rebonds muraux ; Comme nous nous basons sur la vérification des frontières décrite dans le sujet (Effet Trafalgar), nous inversons simplement le vecteur vitesse au lieu de permettre un rebond plus "réaliste".